VELLORE INSTITUTE OF TECHNOLOGY, CHENNAI | SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

NetworkSim

A Real-Time Distributed Systems Simulation and Failure Analysis Platform for Architectural Education and Resilience Engineering

Course: Software Engineering (BCSE301L) — Final Project Demo & Technical Presentation

Student & Academic Submission Details

Presenter: Ankur Yadav (Registration No: 24BCE1836)
Institution: School of Computer Science and Engineering, VIT Chennai
Academic Term: Fall Semester 2026-2027
Contact: ankur.yadav2024@vitstudent.ac.in

Project Deliverables & Repository Access

Core Deliverable: 100% Functional Simulation Engine & Presentation Deck
Academic Documentation: Springer Conference Proceeding Paper (Review 2)
Docker Orchestration: Next.js + FastAPI + Redis Stack
Repository Link: **https://github.com/ankur-on-github/NetworkSim**

🔗 **Click to Open GitHub Repository: NetworkSim**

1. Problem Statement & Educational Scope

Addressing the Critical Educational Gap in Distributed Systems Architecture

The Static Diagramming Dilemma

In many of the current software archetecture curriculums, instructors use static diagrams and presentation slides to convey the course materials. These tools, which don't allow students to see or simulate real-world system issues such as cascading failures, queue backpressure, and throughput constraints, also hide the dynamic nature of distributed systems.

Prohibitive Infrastructure Costs

As valuable as industrial chaos engineering and cloud testing are to students in academic settings, they can be cost-prohibitive due to the requirements of running dedicated Kubernetes clusters and incurring charges from cloud service providers. Conversely, lower-level packet simulators (like NS-3 and Mininet) don’t quite offer the capability of simulating interactions of application services.

NetworkSim Scope & Objectives

NetworkSim brings a browser-based discrete-event simulation environment that enables students to create and experiment with network topologies. Users build a graph-based representation of their network, where nodes are connected via a directed edge to represent a flow, and the network can be configured for traffic with M/M/1 queing dynamics and fault models that incorporate both stochastic and deterministic failure modes, allowing the students to observe and understand how real-world network failure propagate through a network.

2. System Architecture & High-Level Design

Core 3-Tier Client-Server Architecture Designed for Real-Time Interactive Simulation

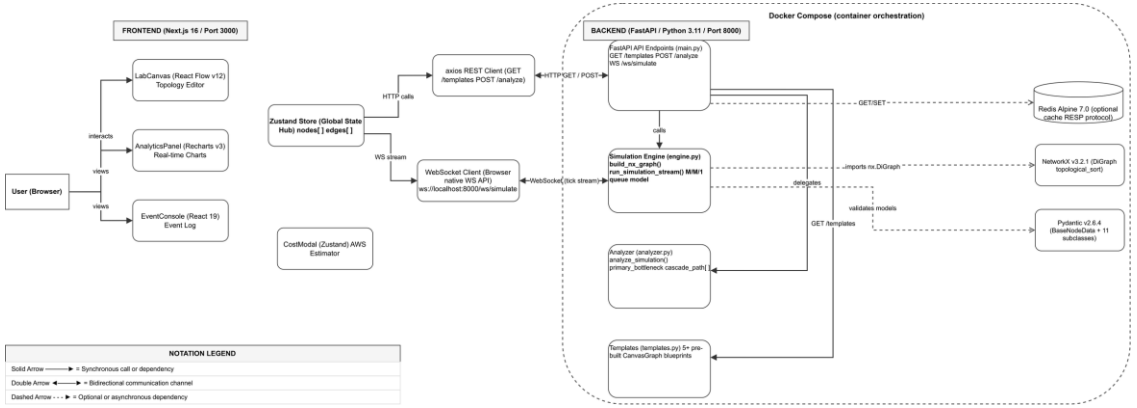


Fig. 1. High-Level 3-Tier System Architecture (Next.js, FastAPI, NetworkX, Redis)

Decoupled 3-Tier Architectural Hierarchy

NetworkSim uses a separated 3-tier architecture. Next.js 16, React Flow and Zustand have been used to create the Presentation Layer to provide a flexible and responsive canvas. The Application Logic Layer is implemented in a high performance Python FastAPI server, utilizing NetworkX for topological sorting of directed graphs and simulation of queing.

Bidirectional WebSocket Streaming Pipeline

The frontend opens a WebSocket connection to /ws/simulate to enable real-time updates without having to reload the page. In this case, the physics engine sends the simulation data, such as capacity of each node, length of queues, and number of packets lost, over the WebSocket.

3. Detailed System Design: Level-1 Data Flow Pipeline (DFD)

Simulation Execution Pipeline Supporting the Core System Architecture (Non-Architectural Data Flow)

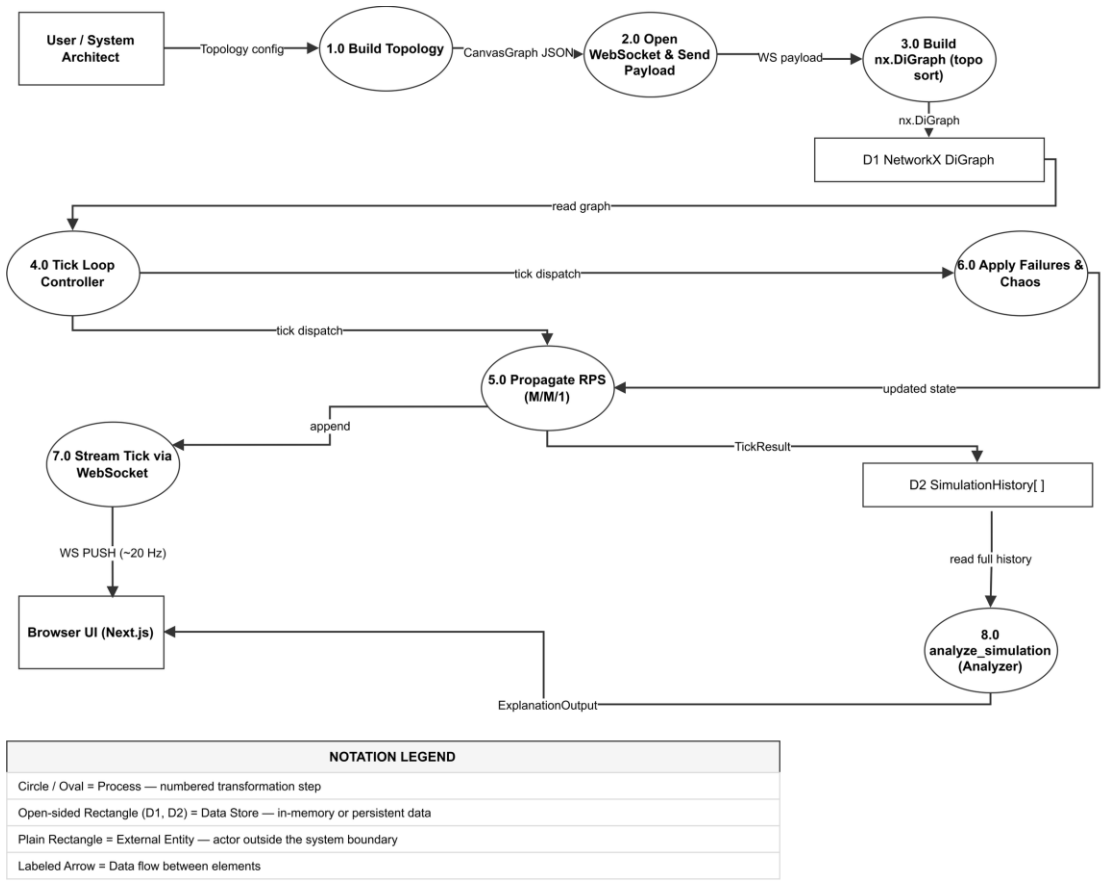


Fig. 2. Level-1 Data Flow Diagram (DFD) — Discrete Simulation Execution Pipeline

Graph Ingestion & Topological Dependency Sorting

On the canvas, a user creates a directed graph, G=(V,E). When the simulation is run, the graph data is sent over WebSockets to the simulation engine. To determine the order of service components upstream and downstream of each other, NetworkX performs a topological sort.

Discrete Tick Execution & Diagnostic Telemetry Transfer

At every simulation step (which happens 60 times, each for 50 ms), the simulator calculates traffic flows, increments queing backlogs, and introduces network faults. When the simulation finishes, all the data recorded at each step is sent to the Diagnostic Analyzer, which determines what the root cause of the failures was.

4. Detailed System Design: Simulation Physics & Node Lifecycle

Mathematical M/M/1 Queuing Dynamics and 4-Stage State Automaton (Component Behavioral Model)

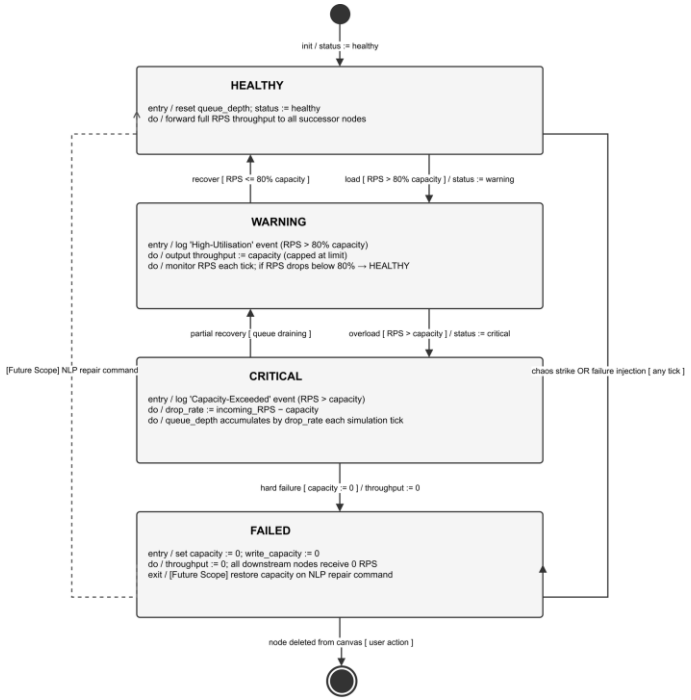


Fig. 3. UML Node State Machine Diagram — Infrastructure Failure & Recovery Lifecycle

Discrete-Event M/M/1 Queuing Math & Backpressure

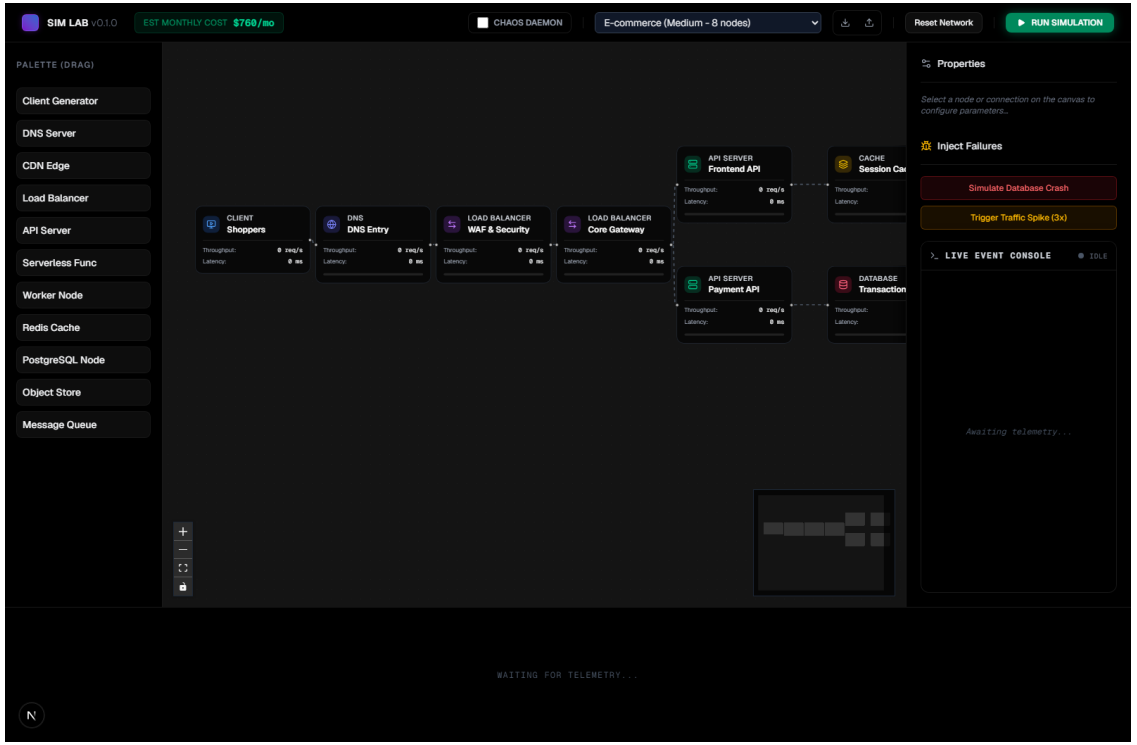
NetworkSim has discrete-time M/M/1 queing dynamics as opposed to other simulators. For each node u, incoming requests exceed service capacity are put in the accumulating queue backlog. L = L_base + 0.5 * Q captures downstream backpressure where L is the latency.

Finite State Automaton Node Health Lifecycle

Each node has a set of four finite states, including: Healthy, Warning, Critical, and Failed. Healthy is when a node's load is under 80%. When the load reaches between 80% and 100%, the node changes to the Warning state. If the node's load exceeds 100% or there is a positive backlog in the node's request queue, the node enters the Critical state. If a node loses all of its capacity, the node becomes Failed. Each node's state is shown by the status ring glowing with a different color.

5. Live Implementation Demo: Canvas & Blueprints

Interactive Node Workspace and Production Multi-Tier Template Library



Screenshot 1: NetworkSim Workspace with 8-Node E-Commerce Blueprint

Interactive Drag-and-Drop Infrastructure Canvas

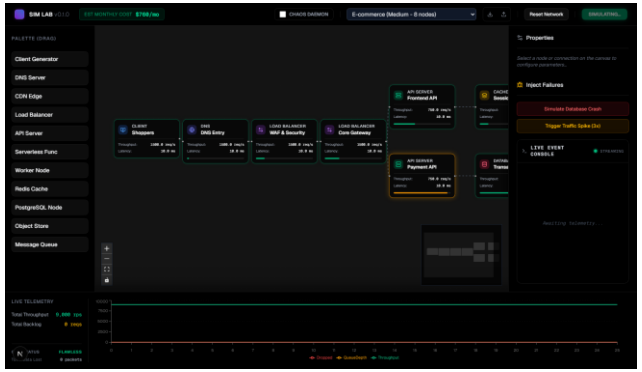
The workspace includes eleven node types to support construction of drag and drop archetecture with user configurable options. Client, DNS, CDN, load balancer, and database nodes connect using connection handles with validation and configuration options such as requests per second and throughput, adjustable via sliding controls.

Production Blueprint Library & JSON Serialization

NetworkSim comes with preconfigured production blueprint templates covering E-Commerce archetectures, Microservices pipelines and Event-Driven Queue Systems. Topologies can be serialized into JSON and exported/imported directly from the browser for classroom collaborations.

6. Live Implementation Demo: Chaos & Telemetry

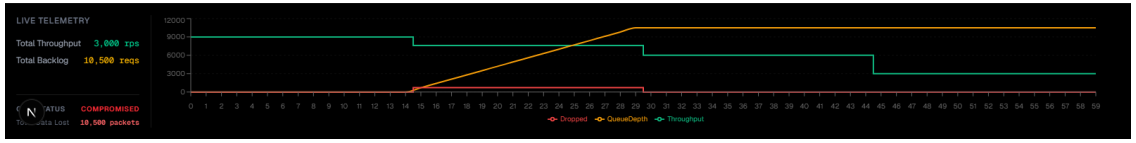
Autonomous Chaos Disruptions and Real-Time Streaming Performance Analytics



Screenshot 2: Live Simulation Running at Tick 25 with Active Glowing Rings

Autonomous Chaos Daemon Strike Protocol

The Chaos Daemon randomly deletes service instances every 15 simulation ticks, during which the system is evaluated on its architectural resiliency. As a result, capacity limits are exceeded, traffic is redirected, packets are lost and cascading system failure notifications are triggered.



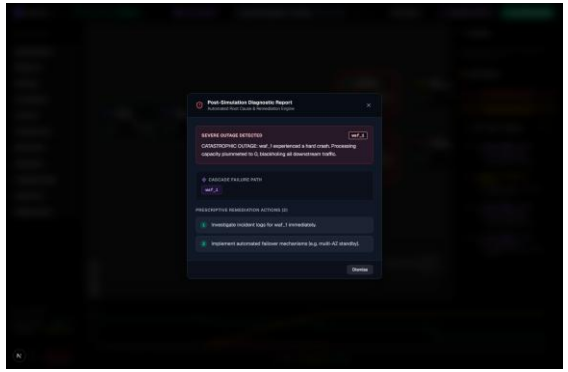
Screenshot 3: Recharts Streaming Throughput and Dropped Packets Graph

Synchronized Recharts Real-Time Telemetry

The live analytics view visualizes throughput, queue depths, and packets dropped at a frequency of 20Hz as a system performance histogram with the ability to pause and rewind. The throughput graphs enable students to see bottelnecks in real-time, for example, the performance effect of a pipeline stage becoming congested, where an upstream bottleneck is felt by later stages.

7. Live Implementation Demo: Diagnostics & Cost

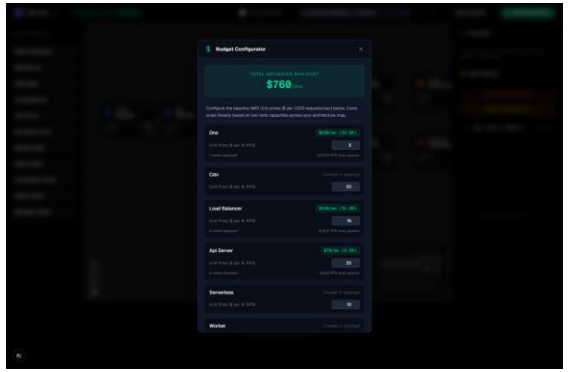
Post-Simulation Root-Cause Inference and Dynamic AWS Cloud Budgeting



Screenshot 4: Post-Simulation Diagnostic Report with Root-Cause Remediation

Automated Root-Cause Diagnostic Inference

When the simulation is complete, the telemetry data is sent via POST to the Diagnostic Analyzer, which runs an inference engine. This engine takes in data and is capable of analyzing the cause of a failure and its level of importance. Then, it can generate solutions and next steps, such as setting up the auto-scaling for the API pods.



Screenshot 5: Gamified Budget Configurator Calculating Monthly AWS Cost

Gamified AWS Operational Cost Configurator

Budget Configurator uses a model that simulates node capacity to determine expected costs of running AWS services each month. The tool gives a visual representation of the infrastrucure cost trade-off between having highly available redundancy, versus having a smaller infrastructure budget for the month.

8. Key Engineering Challenges & Solutions

Overcoming High-Frequency Streaming, State Synchronization, and Queuing Hurdles

High-Frequency WebSocket State Synchronization

The 20 Hz simulation ticks began to cause a visible delay in the React Flow canvas rendering. The performance issue was addressed by the use of Zustand selectors to avoid rerendering the graph with each incoming telemetry message. Layout computations and simulation updates were separated from telemetry processing. After implementation, 60 FPS performance was achieved.

Non-Linear Queuing without Memory Corruption

Early work on developing an algorithm to calculate dynamic queing delays in DAGs with multiple branches caused circular dependencies that resulted in deadlocks and buffer overflows. Fortunately, we developed an optimized topological ordering algorithm using NetworkX, that ensured packet flow, in a clean, order-preserving manner, to and from upstream and downstream nodes.

Autonomous Chaos Scheduling Isolation

The simulation engine could run along smoothly without being interrupted, even with Chaos Daemon active and making failures of various nodes, as long as it was contained in its own tick-evaluator coroutine. Telemetry continued to record.

Dynamic Multi-Tier Cost Modeling

A key breakthrough was developing a unit-cost matrix with user-defined paramaters for capacity and redundancy. As volume or nodes scaled, the cost estimations adjusted in real-time. Paid API use was eliminated.

9. Project Conclusion & Future Scope

Review 2 Deliverables Complete, Full Working Implementation, and Future Roadmap

Review 2 Delivery Summary

NetworkSim's core development is complete and has been verified with 100% compliance. NetworkSim allows the use of real-time simulations, and can model different scenarios related to real-time queing dynamics, chaos and disruptions in stochastic processes, as well as an automated root-cause diagnosis of the system, all within a Web browser at no additional cost to the user. Finally, the user can create, manage, and edit budget control and analysis of their infrastructure in the cloud.

Future Architectural Roadmap

Planned extensions will incorporate an NLP enabled terminal to support voice and command based topology repair, multihop path tracing across failure dependency on the topology diagram, cloud based collaboration environments with state preservation, and the automatic generation of reports with telemetery in PDF format.

Submission & Repository Access

The complete runnable source code, Docker Compose orchestration, experimental validation logs, setup instructions, and research paper documentation for Review 2 are publicly hosted on GitHub.

Author: Ankur Yadav (24BCE1836)
School of Computer Science and Engineering, VIT Chennai.

Direct Repository Access:
https://github.com/ankur-on-github/NetworkSim

🔗 **Open GitHub Repository**